League of Legends: Victory Prediction Using Machine Learning

Andrew Lee

Esports Department, College of William and Mary

SPCH 411: Independent Studies

Dr. Michele King

December 5, 2022

Abstract:

Predicting an esports game's win or lose is not easy and requires extensive analysis. Fortunately, machine learning algorithms may help create insights from the game data. This project focusses on predicting High Elo (Diamond Ranked) League of Legends game results only looking at its first 10 minutes. Visualization and feature selection is done to filter out variables that does and does not impact the prediction. Prediction is done through various classification methods (Random Forest, Logistic Regression and K-Nearest Neighbor). The performance metric used for this project is accuracy.

For the coding, please refer to the following link: https://github.com/leeanddrew

Section I: Introduction

Introduction:

With the esports industry rapidly growing, field of analytical esports has also taken its root. League of Legends, commonly referred to as league, is one of the most popular MOBA (Multiplayer Online Battle Arena) game played around the world. There are about 150 million registered players, and million players are actively playing monthly. The game also has numerous professional tournaments around the globe and a World Championship where each country's league's top team compete each other. With the increasing service of online streaming platforms such as Twitch and YouTube, League of Legends is not only a game to play, but a game to watch. Thus, like any other sports, victory prediction has become popular amongst esports analytics.

The main aim of victory prediction is to provide useful insights for both players and coaches to improve their performance on. However, there are restrictions that prevent us from making accurate predictions. Like any other sports, League of Legends is played by ten humans. Predicting a human vs. human match is not easy as there involves variables unanalyzable by machines (individual conditions, teamwork, luck/mistakes).

League of Legends is a great place to apply various machine learning models with its voluminous amount of data. For example, Valmiro Ribeiro da Silva has utilized database of biometric information to create a neural network algorithm for identifying different users on the same account. This could potentially prevent Elo Boosting and account fraud not easily detected by current methods.

This project focusses on predicting victory in Diamond Ranked matches by their 10 minutes features. Dataset comprises of 9878 games each game having 39 independent variables and 1 dependent variable. With the help of visualizations and machine learning algorithms, we will hopefully achieve reliable prediction accuracy.

Game Description:

League of Legends is a MOBA game. The game consists of two teams of five players, where they compete to destroy a structure named Nexus. Unlike traditional MMORPG (Massively Multiplayer Online Role-Playing Game), League of Legends relies on well-designed strategies and cooperative team play and individual character development.

Section II: Data Description/Visualization

Data Source:

This data was adapted from Kaggle (open-source database) by Yi Lan Ma. The dataset consists of first 10 minutes statistics of approximately 10,000 solo-queue ranked games from high-Elo (Diamond I to Master). The statistics include, but not limited to, each team's kills, deaths, gold, experience, level, etc.

Pre-visualization:

The data includes both blue and red team's performance. To reduce the complexity of our model, we will remove all variables regarding red team.

Visualizations:

Figure 1 shows a pie chart of the number of observations regarding victory and loss. It is evident that there about equal number of games where blue team wins and losses. This is critical for classification model, because a skewed population of data can negatively alter the metrics of interest.

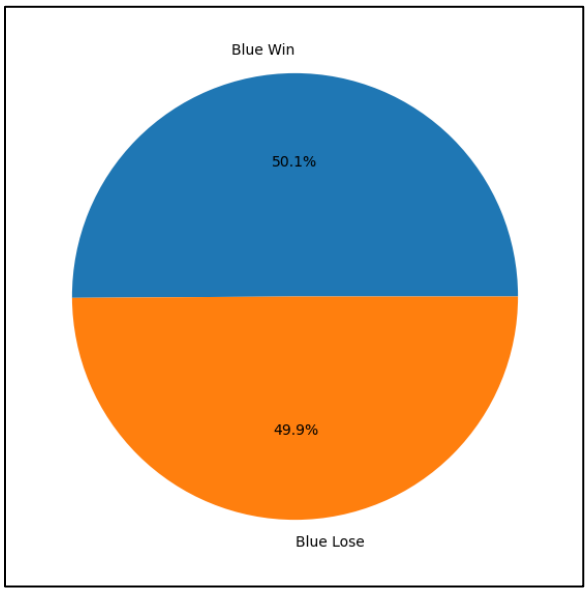

Figure 1: Pie Chart of Observations

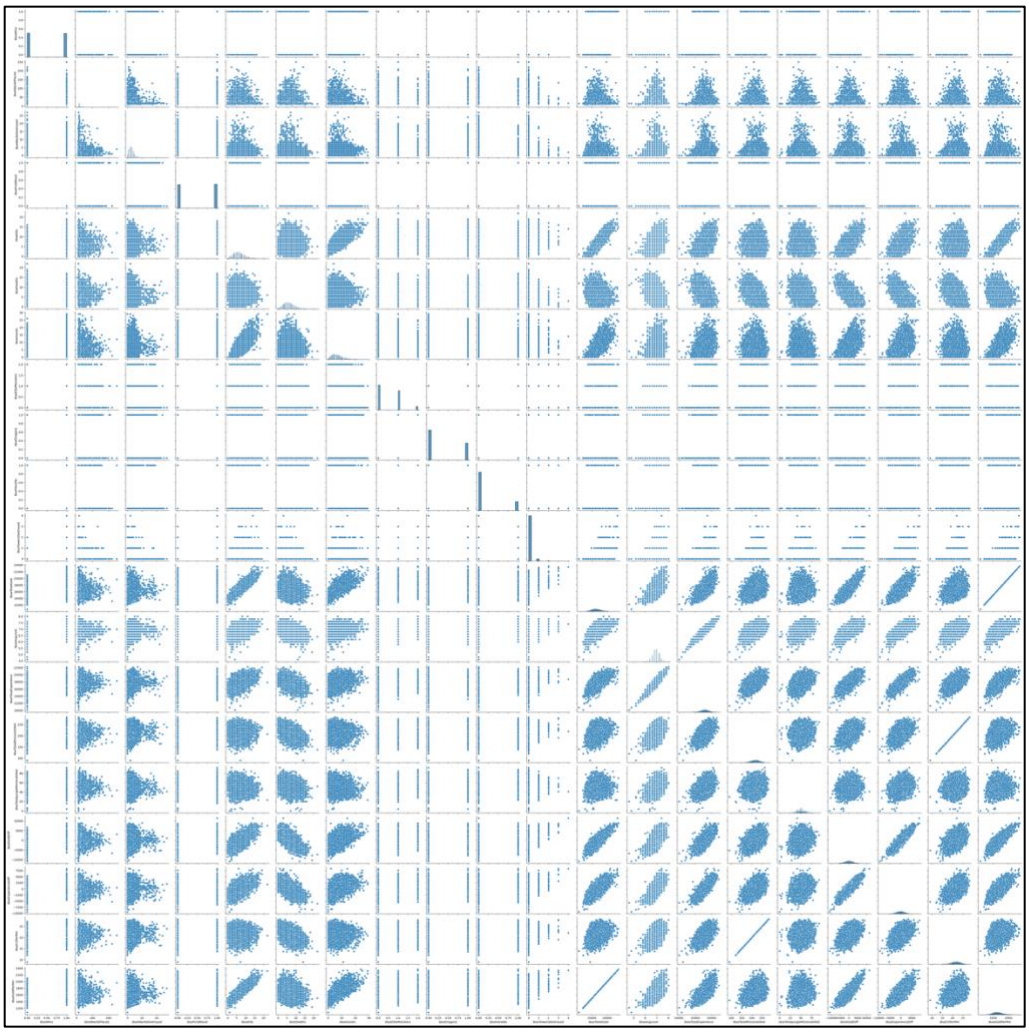

Figure 2: Pair Plot of All Variables

The pair plot in *Figure 2* is hard to read due to the large number of variables we have. However, we see that most numerical variables have a linear relationship with each other. We will form a heatmap to see the exact correlation coefficients between variables.

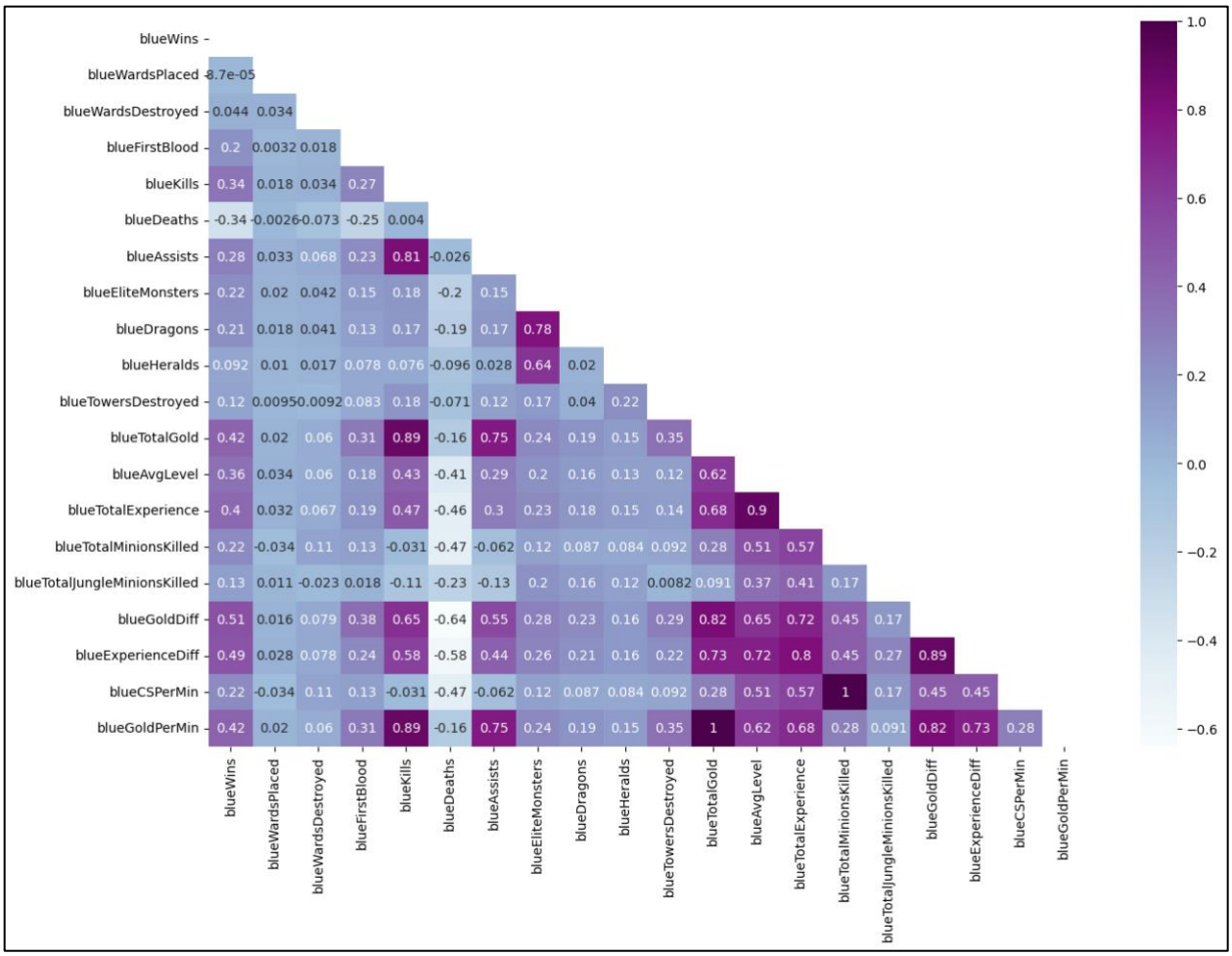

Figure 3: Heatmap of Correlation Coefficients

According to the heatmap in *Figure 3*, most highly correlated variables seems to be total experience and average level. Also kills and total gold/gold per minute is highly correlated. This makes sense as getting kills gets you more gold. We also see high correlation between gold difference and experience difference.

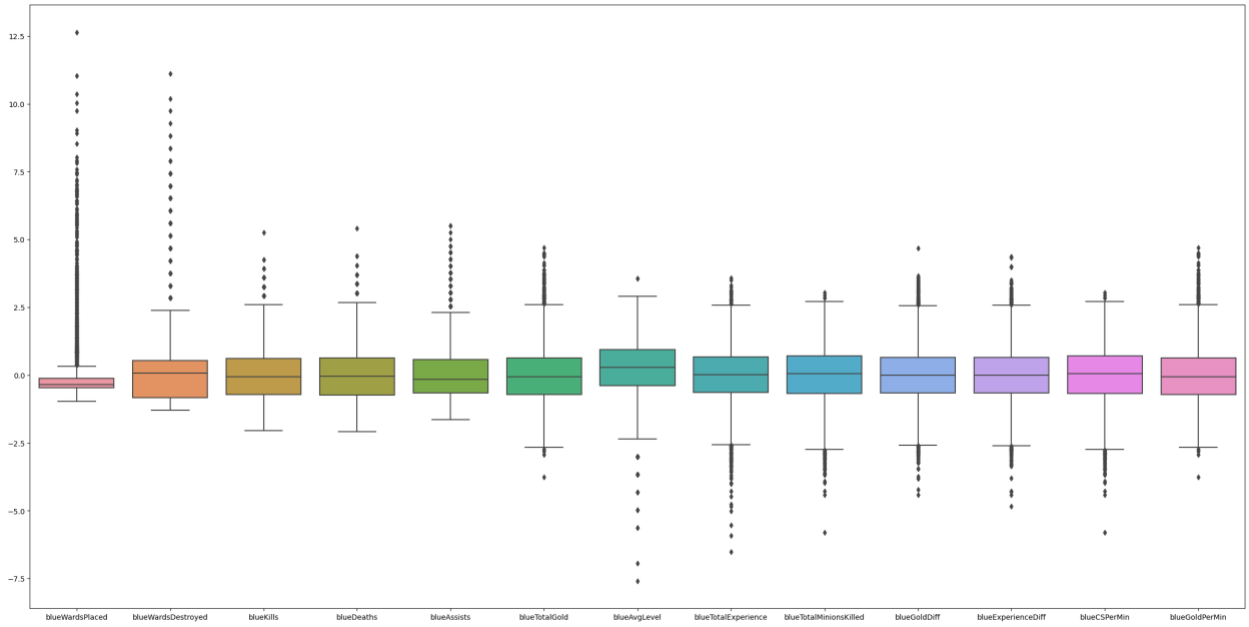

Figure 4: Boxplot of Variables (Scaled)

After scaling the data using scitkit-learn's standard scaler, we can clearly see the boxplot of each variable in *Figure 4*. We may initially think that there are too many outliers in each variable. However, note that total of 10,000 observations are in the dataset. Considering the large amount of data we have, the number outliers shown in the boxplot isn't that significant. Overall, we can conclude that most of the numerical variables follow an approximately uniform distribution.

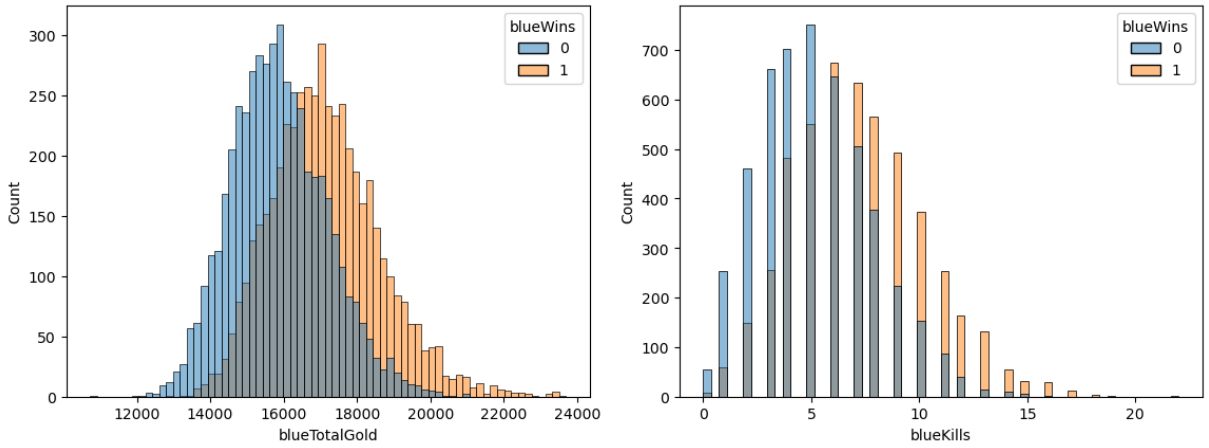

Figure 5: Histogram of Total Gold and Kills

Figure 5 above depicts the histogram of total gold and kills with respect to victory and loss (0 indicates loss and 1 indicates victory). Interestingly, the distribution of victory and loss seems to be similar (uniform), but just shifted. Intuitively, it makes sense since more gold allows players to buy better items which increases the chance of killing enemies. Similarily, we have more games where blue team won with more kills than less kills. Therefore, we can conclude that two variables will be very helpful when predicting victory.

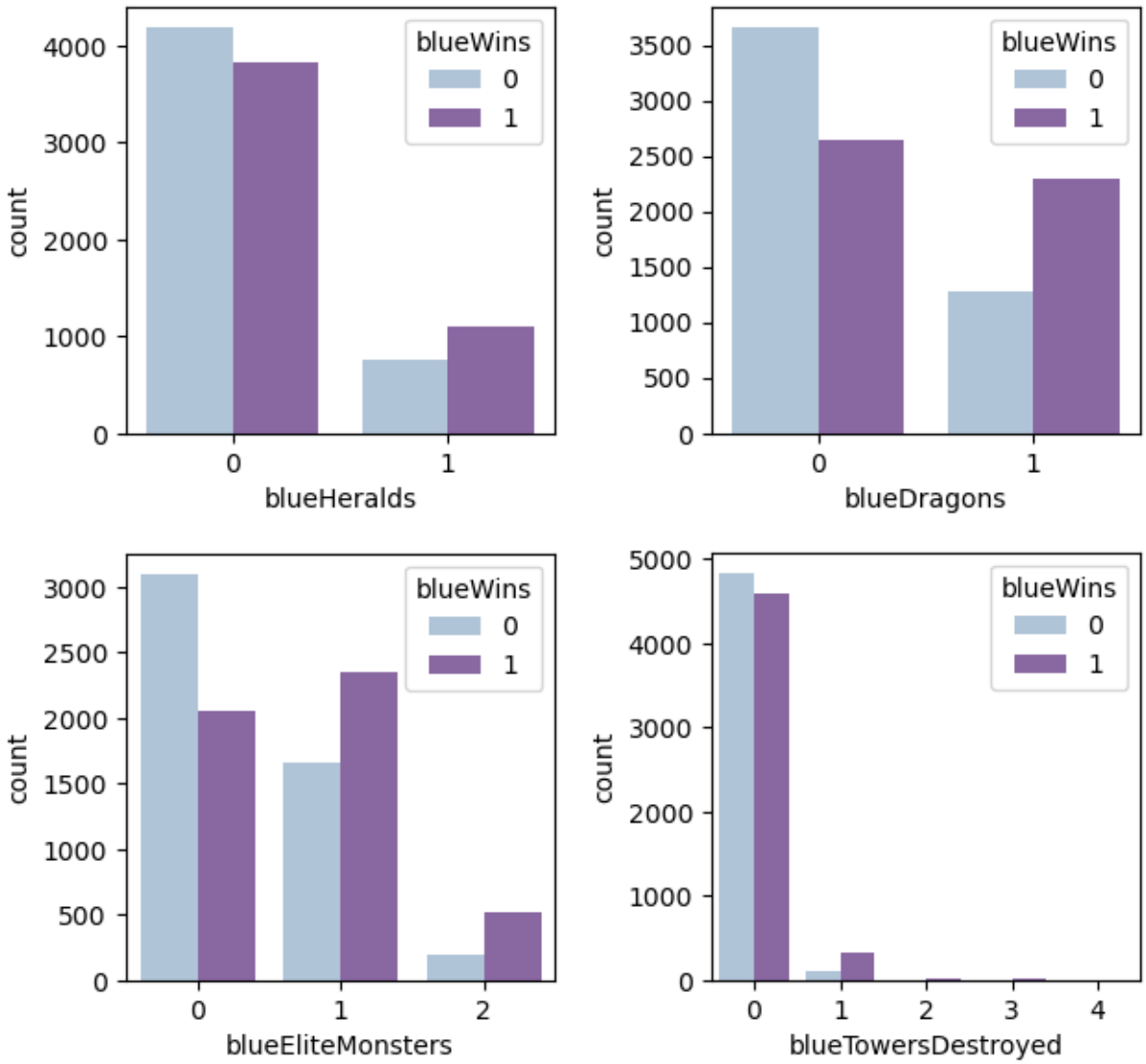

Figure 6: Histogram of Objectives Secured

Figure 6 histogram shows whether a relationship exists between securing objectives and the game outcome. Securing dragons and elite monsters seems to lead to higher chance of winning. Blue team won more games when they secured dragons and elite monsters in the first 10 minutes.

Section III: Modeling and Preparation

Data Cleaning / Preparation:

Before feeding the data into classification model, we want to turn categorical variables into dummy variables. Using pandas get dummies function, I turned categorical variables (elite monsters, heralds, dragons, towers) into dummy variables. By creating dummy variables, we are

turning categorical variables into numerical representation (e.g. 0 representing ‘no’ and 1 representing ‘yes’).

Modeling:

In this project, I have used total of 3 machine learning models to fit the data: Logistic Regression, Random Forest Classifier and KNN Classifier). By comparing the different models, we will evaluate which one to use at the end.

Logistic Regression

```
In [254]: from sklearn.linear_model import LogisticRegression
          from sklearn.model_selection import train_test_split

In [255]: from intro_Data import do_Kfold

In [256]: ss = StandardScaler()

In [257]: log_reg = LogisticRegression()

In [271]: train,test = do_Kfold(log_reg,X,y,k=10,scaler=ss)

In [275]: np.mean(test)

Out[275]: 0.7308452186111761
```

Figure 7: Logistic Regression

Figure 7 shows the code for logistic regression. With scaling the data and using K-Fold validation, we were able to yield an average test score of approximately 0.7308. This means that, on average, we will predict 73% of the game’s results correctly given the inputs.

Random Forest Classifier

```
In [261]: from sklearn.ensemble import RandomForestClassifier
          from sklearn.model_selection import GridSearchCV

In [262]: rfc = RandomForestClassifier()

In [263]: param_grid = {'n_estimators':[100,200,500],'max_depth':[2,3,4,5],'min_samples_split':[2,3,4,5]}

In [264]: grid = GridSearchCV(rfc,param_grid,scoring='accuracy',cv=5)

In [265]: grid.fit(X,y)

Out[265]: GridSearchCV(cv=5, estimator=RandomForestClassifier(),
                       param_grid={'max_depth': [2, 3, 4, 5],
                                   'min_samples_split': [2, 3, 4, 5],
                                   'n_estimators': [100, 200, 500]},
                       scoring='accuracy')
          In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
          On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [267]: print('best parameters:',grid.best_params_)
          best parameters: {'max_depth': 5, 'min_samples_split': 3, 'n_estimators': 500}

In [268]: results = pd.DataFrame(grid.cv_results_)[['param_n_estimators','param_max_depth',
                                    'param_min_samples_split','mean_test_score','rank_test_score']]

In [270]: results[results['rank_test_score'] == 1]

Out[270]:
```

	param_n_estimators	param_max_depth	param_min_samples_split	mean_test_score	rank_test_score
41	500	5	3	0.72973	1

```
In [276]: rfc = RandomForestClassifier(n_estimators=500,max_depth=5,min_samples_split=3)

In [277]: train_rfc, test_rfc = do_Kfold(rfc,X,y,k=10)

In [281]: print(np.mean(test_rfc))
          0.7269988596696322
```

Figure 8: Random Forest Classifier

Figure 8 shows the result of a random forest classifier model. Surprisingly, it did worse than logistic regression with average score of 0.7270. Regarding its computational burden and complexity, we probably will want to avoid using random forest classifier for this dataset.

KNN Classifier

```
In [282]: from sklearn.neighbors import KNeighborsClassifier

In [284]: knn = KNeighborsClassifier()

In [283]: param_grid_knn = {'n_neighbors':range(2,40),'weights':['uniform','distance']}

In [285]: grid_knn = GridSearchCV(knn,param_grid_knn,scoring='accuracy',cv=10)

In [286]: scaler = StandardScaler()
          X_scaled = scaler.fit_transform(X)

In [287]: grid_knn.fit(X,y)

Out[287]: GridSearchCV(cv=10, estimator=KNeighborsClassifier(),
                       param_grid={'n_neighbors': range(2, 40),
                                   'weights': ['uniform', 'distance']},
                       scoring='accuracy')
          In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
          On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [288]: print(grid_knn.best_params_)
          {'n_neighbors': 31, 'weights': 'uniform'}

In [289]: knn = KNeighborsClassifier(n_neighbors=31,weights='uniform')

In [290]: train,test = do_Kfold(knn,X,y,k=10,scaler=scaler)

In [293]: print(np.mean(test))
          0.7185967168329991
```

Figure 9: KNN Classifier

Figure 9 shows the result of a KNN classifier model. So far, KNN produced the worst result with average score of 0.7186.

Model of Choice:

Considering the computation burden and complexity, logistic regression seems to be winner without a doubt. It produced an accuracy of 73% which isn't too bad considering the difficulty to predict a game's outcome.

Section IV: Discussion and Conclusion

Discussion:

Obviously, the model is not perfect. We cannot be certain that we will predict most games' outcome from a 73% accuracy. There are external factors that cannot be determined with the data such as luck, mistakes, teamwork, adrenaline, etc. Therefore, a perfect model cannot exist for this project, but we can try improving our methods. Because we have approximately 10,000 observations, we may want to try deep learning using artificial neural networks. ANNs can handle large number of inputs and nonlinear data. Also, they tend to work well with binary classifications using sigmoid function activation. Outside of modeling methods, we may want to collect more data that may be significant to predicting the outcome. For example, in high-Elo

games, champion selection may be useful for predicting the game's outcome. By observing the win-rate of each champion (meta) and the counter picks may increase the accuracy of our model.

Conclusion:

In summary, our results demonstrate that first 10 minutes of the game data can be used to adequately predict the results. We saw that total gold and kills serve as an important factor for determining victory or loss of a game. Additionally, securing objectives such as dragons and elite monsters also have correlations with winning. The model produced in this project may be developed into an active learning model to continuously train the model. Also, it can help high-Elo League of Legends players to reflect on the factors of determining victory in the first 10 minutes.